Estabilidade de Treinamento: Inicializações, Normalizações e Regularização

Aula 05 — A Física Interna do Treinamento Estável em PyTorch

Instrutor: **Prof. Allan Spadini**

Redes Neurais Profundas | Graduação Faculdade Infnet

O Colapso de Simetria: O Desastre dos Pesos Nulos

Por que inicializar W = 0 e b = 0 reduz 1.000 neurônios a apenas 1

PASSO 1
Pesos Nulos

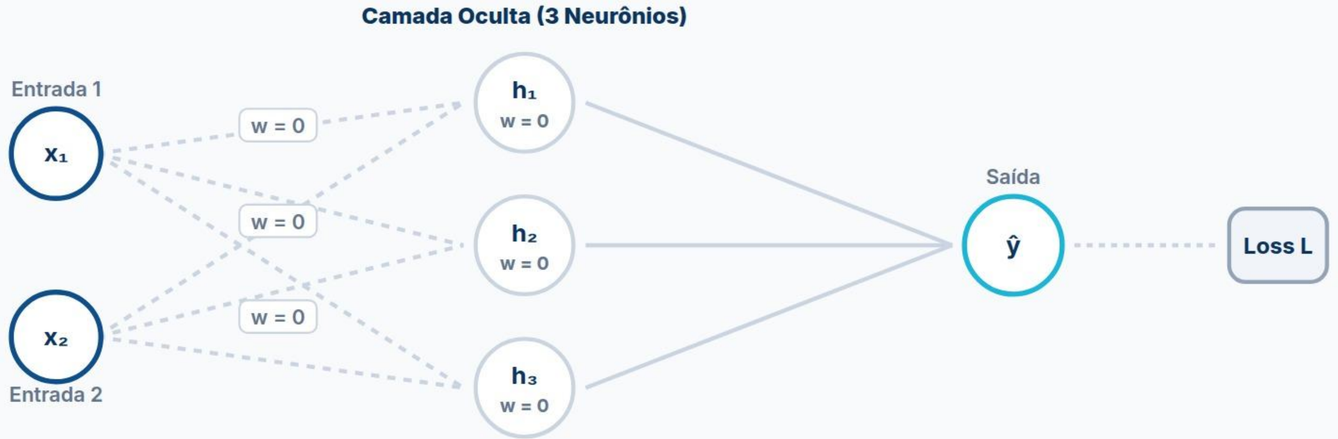
PASSO 2
Forward Idêntico

PASSO 3
Backward Idêntico

PASSO 4
Colapso da Camada

INICIALIZAÇÃO W = 0 **Todos os pesos e bias iniciam exatamente em zero**

⚠ Aguardando fluxo de sinal...



DIAGNÓSTICO TEÓRICO:
Em regressão linear simples y = wx + b, começar em zero funciona. Em redes neurais profundas, todas as sinapses partem com w = 0.0.

→ Avançar Passo

A Quebra de Simetria: Inicialização Calibrada

Como o ruído estocástico controlado viabiliza detectores especializados

PASSO 1
Pesos Diversificados

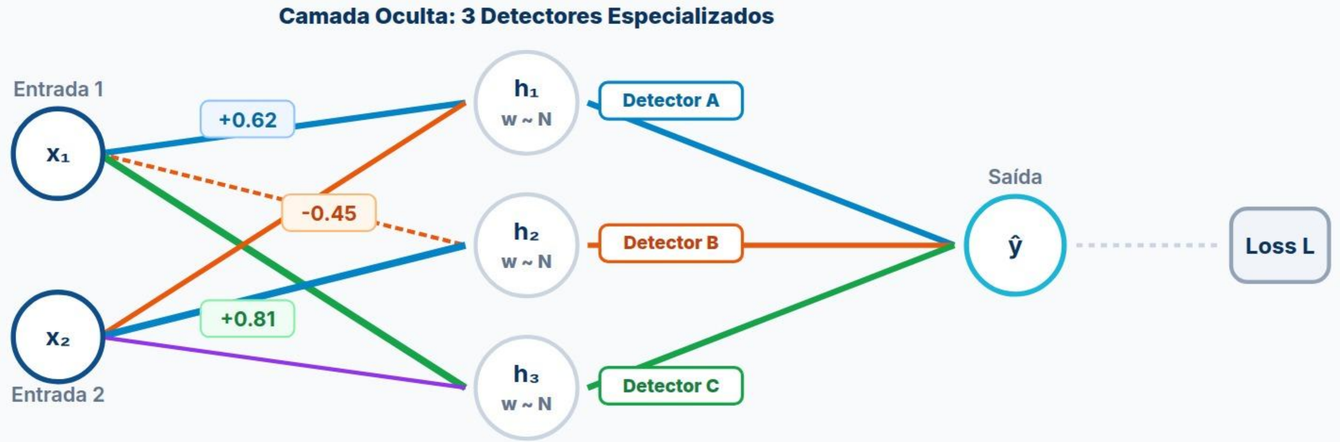
PASSO 2
Forward Especializado

PASSO 3
Backward Independente

PASSO 4
Fronteiras Complementares

RUÍDO GAUSSIANO W ~ N(0, σ²) **Cada sinapse recebe um valor aleatório distinto e calibrado**

Sinapses com orientações únicas



VANTAGEM DA QUEBRA DE SIMETRIA:
Pesos pequenos com sinais positivos e negativos geram quebra imediata de simetria: w_{11} = +0.62, w_{12} = -0.45, w_{13} = +0.81.

→ Avançar Passo

A Matemática de Xavier e Kaiming He

Calibrando a variância dos pesos para manter o fluxo de sinal estável

$$Var(W)_{Xavier} = \frac{2}{n_{in} + n_{out}}, \qquad Var(W)_{He} = \frac{2}{n_{in}}$$

n_{in}

Número de conexões de entrada (fan-in da camada).

n_{out}

Número de neurônios de saída (fan-out da camada).

Xavier (Glorot)

Ideal para funções lineares ou simétricas: Tanh e Sigmoid.

Kaiming He

Desenvolvida para ReLU e LeakyReLU, compensando o fato de que metade das entradas vira zero.

Comparador Interativo de Inicialização de Pesos

Veja a distribuição das ativações através de 5 camadas profundas

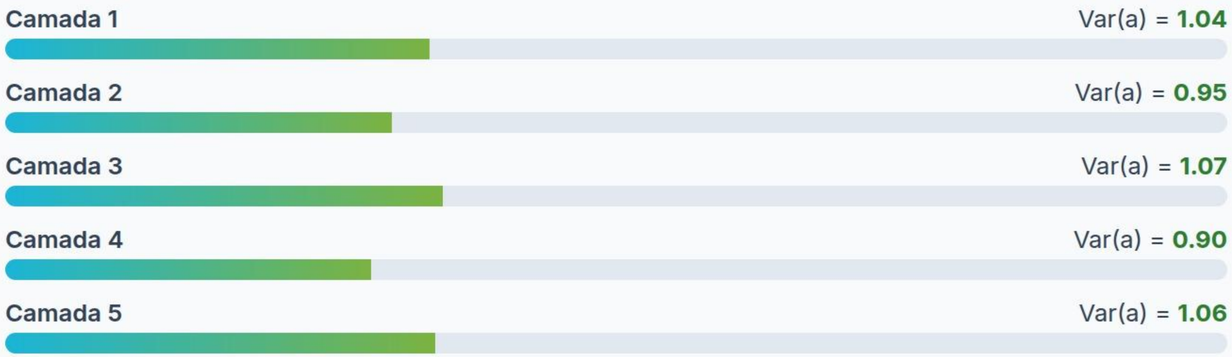
Comparador de Inicialização de Pesos

Zeros | N(0, 1) | Xavier | **Kaiming He** | Sortear Novo Lote

Variância Amostral das Ativações: Var(a)

Meta Saudável: ≈ 1.00

A variância **não tem ordem fixa**; ela flutua realisticamente em torno de 1.0 dependendo das amostras do lote.



✨ Xavier e He evitam que Var(a) exploda para > 100 ou caia para 0.00.

He / Kaiming (Normal)

`Var(W) = 2 / n_in — Ideal para ReLU / LeakyReLU`

Exemplos de Pesos Reais Sorteados (W):

`-0.28` `+0.38` `-0.18` `+0.24` `-0.30`

✓ Os pesos são aleatórios (sem ordem), com média 0.

Recomendado para ReLU: Dobra a Energia Inicial

Compensa a perda de 50% dos sinais ceifados pela ReLU (metade negativa nula), mantendo a variância estável ~ 1.0.

O Fenômeno do Internal Covariate Shift

Por que treinar redes profundas sem normalização é como mirar em um alvo móvel

ETAPA 1
O Alvo Móvel (Sem Normalização)

ETAPA 2
Efeito Cascata em Redes Profundas

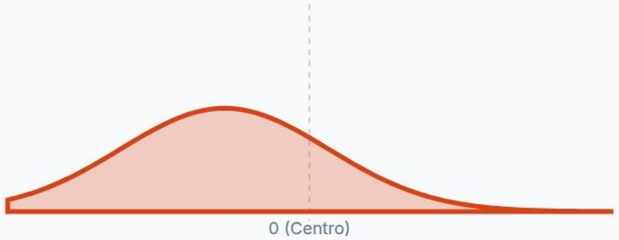
ETAPA 3
A Ancoragem: Normalização Z-Score

ETAPA 4
Restauração de Capacidade (γ e β)

DERIVA INTERNA **1. O Alvo Móvel (Sem Normalização)**

⚠ Instabilidade severa entre épocas

ÉPOCA t (Lote Inicial) μ = -1.8, σ = 2.2



ÉPOCA t + 5 (Após Atualizações) μ = 2.5, σ = 3.8



ANALOGIA PEDAGÓGICA (O ALVO MÓVEL):
Conforme a Camada 1 atualiza seus pesos, a distribuição das suas saídas oscila violentamente. A Camada 2 precisa aprender a perseguir um alvo que nunca para no mesmo lugar.

→ Avançar Passo

BatchNorm1d vs. LayerNorm: Geometria dos Tensores

Duas formas complementares de fatiar o tensor 2D [N Amostras × C Features]

BatchNorm1d (Fatia Vertical / Eixo N) | LayerNorm (Fatia Horizontal / Eixo C)

Tensor 2D de Ativações: **[N Amostras × C Neurônios]**

Eixo das Amostras (Batch N = 4) ↓ — Eixo das Features (C = 4) →

	Feat 1	Feat 2	Feat 3	Feat 4
\mostra #1	x1,1	x1,2	x1,3	x1,4
mostra #2	x2,1	x2,2	x2,3	x2,4
mostra #3	x3,1	x3,2	x3,3	x3,4
mostra #4	x4,1	x4,2	x4,3	x4,4

ı e σ nesta coluna

Cálculo de BatchNorm1d

$$\mu_c = (1/N) * \Sigma_i \, x[i, c]$$

- Normaliza através de **todas as amostras do lote** para uma feature específica.
- Requer batch representativo (ideal ≥ 32).
- Mantém médias móveis (`running_mean`) durante o treino.
- Mais comum em **Visão Computacional e MLPs tabulares**.

Se N = 1 no treino, BatchNorm quebra por variância nula!

O Modo Crucial: model.train() vs model.eval()

A alternância de estado que previne os maiores bugs silenciosos em PyTorch

A Chave Operacional de Estado no PyTorch

Alterne o modo da rede para inspecionar a mutação de comportamento do BatchNorm e do Dropout

model.train() | model.eval()

CAMADA: nn.BatchNorm1d — • Estatísticas Dinâmicas

Cálculo sobre o Mini-Batch Atual

Calcula μ e σ² das amostras do lote em tempo real. Atualiza running_mean e running_var acumuladas.

```
x_norm = (x - mean_batch) / sqrt(var_batch + eps)
```

Gera leve ruído amostral benéfico

CAMADA: nn.Dropout(p=0.5) — ⚡ Máscara Bernoulli Ativa

Estocástico (Neurônios Zerados)

50% dos neurônios são desligados aleatoriamente. Neurônios sobreviventes recebem amplificação de 1/(1-p) = 2.0x.

```
y = (mask * x) / (1 - p) # Inverted Dropout
```

Previne co-adaptação e sobreajuste

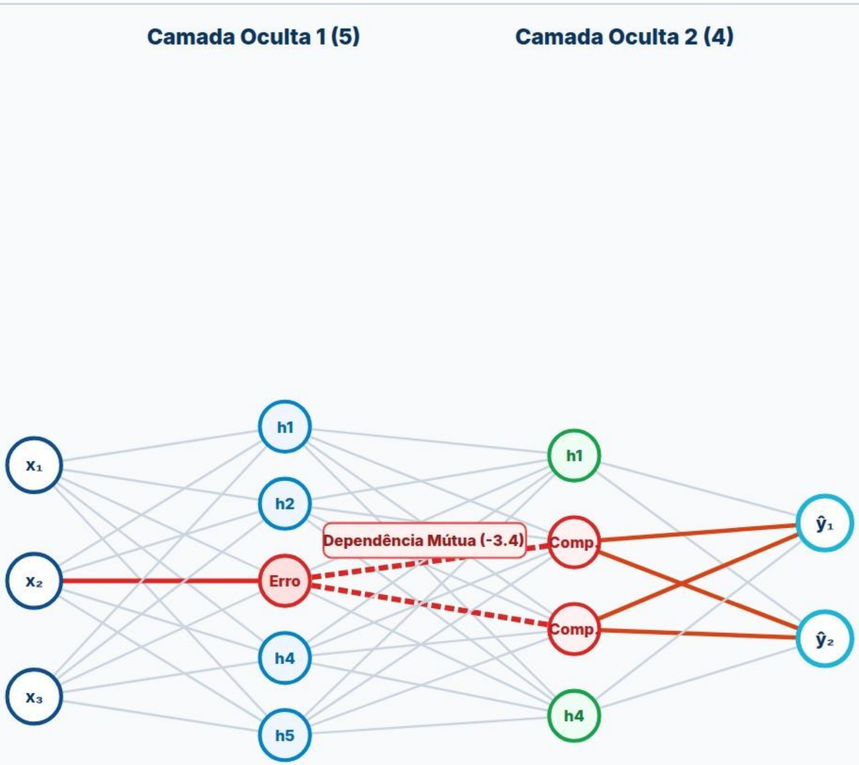
O Bug Silencioso Mais Comum: Se você esquecer de chamar `model.eval()` antes de validar ou testar, o Dropout continuará apagando metade dos neurônios aleatoriamente a cada predição e o BatchNorm falhará em amostras isoladas!

O Que é Co-adaptação de Neurônios?

Como o sobreajuste surge em um Perceptron Multicamadas (MLP)

1. Sem Regularização: Co-adaptação Multicamadas | 2. Com Dropout (p=0.4): Sub-Redes Autônomas

Teia de dependência mútua ativa



A Armadilha do MLP Denso

Em um Perceptron Multicamadas sem regularização, neurônios da Camada 2 aprendem a anular erros específicos cometidos pela Camada 1. Eles não aprendem a essência dos dados, mas sim a mascarar os defeitos mútuos!

Comportamento no Treino vs Teste:

Loss Treino: **0.04** Loss Val: **0.92 (Overfitting!)**

💡 *Dica:* Em MLPs tabulares, use taxas moderadas de dropout ($p \in [0.2, 0.4]$) entre camadas lineares.

A Matemática do Inverted Dropout

Por que o PyTorch reescala os sinais durante o treinamento

$$y = \frac{m \odot x}{1 - p}, \qquad m \sim \text{Bernoulli}(1 - p)$$

m

Máscara binária onde cada elemento é 1 com probabilidade (1-p) e 0 com probabilidade p.

p

Taxa de dropout (ex: p = 0.3 desliga 30% dos neurônios).

1 / (1 - p)

Fator de amplificação para compensar a energia dos neurônios desligados no treino.

Inferência sem Custo

Na avaliação, o PyTorch não precisa multiplicar por nada (y = x direto).

Laboratório Interativo: Dropout & Generalização

Ajuste a taxa de dropout e observe o fechamento do gap de overfitting

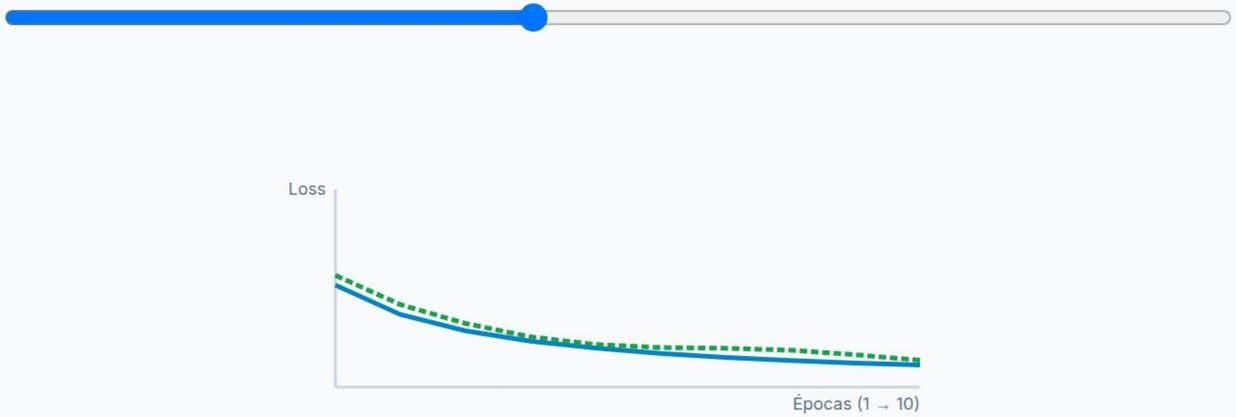
Laboratório: Dropout & Fechamento do Gap de Generalização

Ajuste a probabilidade de descarte (p) e veja a resposta imediata nas curvas de loss

Modo Treino (Ativo) | Novo Lote

Taxa de Dropout: p = 0.30

Escala Inverted: 1.43x



— Loss Treino: 0.13

--- Loss Validação: 0.16 (Gap Fechado ✓)

Estado dos Neurônios Ocultos no Lote Atual:

h1	h2	h3	h4	h5	h6
⚡ 1.43x	⚡ 1.43x	⚡ 1.43x	✕ 0.0	⚡ 1.43x	⚡ 1.43x

p ideal (Equilíbrio & Generalização)

A rede aprende representações robustas e o gap entre treino e teste permanece mínimo.

Estudo de Caso: Classificação no Dataset Iris

Comparando o impacto da estabilização na planilha e em código PyTorch

</> Experimentos PyTorch (Curvas Empíricas de Loss) | Planilha Interativa (simulacao_mlp_iris.xlsx)

Dataset: **Iris (150 amostras, 3 classes)**

Sem Normalização | Com BatchNorm1d | **BatchNorm + Dropout (0.3)**



Loss Final: 0.05

Velocidade: 25 épocas

MELHOR GENERALIZAÇÃO **98.3% Acerto**

Combinação perfeita: BatchNorm garante gradientes estáveis enquanto Dropout fecha o gap de validação sem sobreajustar.

Conclusão Científica:

Normalizar com BatchNorm1d triplicou a velocidade de convergência e o Dropout evitou que a rede memorizasse ruídos das amostras de treino.

Quiz de Fixação: Estabilidade e Regularização

Teste seus conhecimentos sobre simetria de pesos, BatchNorm e Dropout

Questão 1 de 4

Pontuação: **0** / 4

1. O que ocorre se inicializarmos todos os pesos e bias de uma MLP com zero (W = 0, b = 0)?

A) Colapso de Simetria: todos os neurônios calculam saídas idênticas, recebem gradientes iguais e a camada age como 1 único neurônio.

B) O treinamento converge mais rápido, pois o ponto inicial tem perda zero.

C) O PyTorch dispara automaticamente um erro de divisão por zero.

D) A rede aprende normalmente, contanto que a taxa de aprendizado seja alta.